

Краткая вводная по инфраструктуре «Протоколы сборки лаборатории ХИТ»

Дата: 25.05.2026

Статус: проект для согласования

Ответственный исполнитель / функциональный владелец / руководитель разработки:

Мараулайте Д.К.

Соработчик / технический исполнитель по отдельным модулям, Vue/V2, инфраструктуре:

Меняйлов Д.С.

Электрохимическая экспертиза: Баталов Р.С. при необходимости.

Основание: вводная для совещания с ИТ/Тринатомом по размещению решения на ресурсах Росатома.

«Протоколы сборки лаборатории ХИТ» можно развернуть как обычное внутреннее веб-приложение: PostgreSQL + Node/Express + браузерный интерфейс. Для пользователей достаточно доступа через браузер, в том числе через терминальный сервер. Прямой доступ к БД должен быть оставлен только администраторам/сопровождению. Текущая vanilla v1 версия готова для лабораторного пилота; Vue/V2 следует рассматривать как следующий интерфейс, который должен сохранить правила данных и workflow текущей рабочей версии.

1. Цели и задачи разрабатываемого решения

«Протоколы сборки лаборатории ХИТ» — рабочее наименование разрабатываемого решения для сопровождения разработки и сборки аккумуляторных элементов. Техническое обозначение в коде и БД: **BADB** (*Battery Assembly Database*).

По назначению система относится к классу лабораторных информационных систем (ЛИС/ЛИУС; англ. LIMS), но рабочее наименование разрабатываемого решения: «Протоколы сборки лаборатории ХИТ».

Основные задачи:

- фиксировать технологические данные по материалам, рецептам, электродным лентам, партиям электродов, сепараторам, электролитам и аккумуляторам;
- связать между собой исходные материалы, технологические операции и итоговые аккумуляторные элементы;
- снизить риск потери лабораторных данных и ошибок ручного учёта;
- обеспечить прослеживаемость: из какого материала, рецепта, ленты и партии электродов собран конкретный элемент;
- формировать печатные отчёты/протоколы по ключевым сущностям;
- подготовить базу для последующего расширения: испытания, графики, инвентаризация материалов, протоколирование других лабораторных процессов (синтез, химический анализ), аналитика, отчётность и более развитый веб-интерфейс.

Текущая версия, планируемая к использованию в лабораторном пилоте, — vanilla v1/v1.1. Версия V2 на Vue рассматривается как следующий фронтенд-слой после синхронизации с правилами и поведением текущей рабочей версии.

2. Архитектура решения в целом

Решение состоит из следующих основных компонентов:

- PostgreSQL — основная реляционная база данных и источник истины;
- Node.js/Express — серверное приложение и API под `/api/`;
- статический веб-интерфейс vanilla v1 в папке `public/`;
- Vue/Vite фронтенд в папке `client-web/`, предназначенный для версии V2;
- файловое хранилище для загружаемых лабораторных файлов;
- SQL-миграции `migrations/` и ASCII-зеркало `migrations_ASCII/`, используемые при обновлении и расширении структуры PostgreSQL;
- служебные скрипты проверки, резервного копирования и восстановления (`contract`, `smoke`, резервное копирование/восстановление).

Общий поток работы:

пользователь в браузере -> веб-интерфейс ->

Express API -> PostgreSQL ->

ответ API -> обновление интерфейса / отчёта

Обычные пользователи работают через браузер. Прямой доступ к PostgreSQL нужен только администратору/разработчику для обслуживания, резервного копирования, миграций и восстановления.

3. Архитектура фронтенда

Текущий рабочий интерфейс vanilla v1 реализован как набор HTML/CSS/JavaScript страниц:

- `public/workflow/` — основные лабораторные процессы;
- `public/reference/` — справочники;
- `public/js/` — логика страниц;
- `public/css/` — стили.

Текущий vanilla-интерфейс уже поддерживает:

- ввод и редактирование лабораторных сущностей;
- списки, фильтры и карточки записей;
- печатные отчёты по ключевым объектам;
- безопасные сценарии удаления/проверки зависимостей;
- расчётные подсказки для рецептов, электродов и аккумуляторов;
- авторизованную работу пользователей.

Параллельно существует Vue/Vite фронтенд в `client-web/`. Он использует Vue 3, Vue Router, Pinia, PrimeVue, Axios, Chart.js/Cytoscape для будущего более развитого интерфейса и аналитики. В текущем плане vanilla v1 является рабочей пилотной версией, а Vue/V2 должен быть приведён к функциональному и поведенческому паритету по данным перед заменой рабочего интерфейса.

4. Архитектура бэкенда, базы и аналитики

Бэкенд:

- Node.js;
- Express;
- PostgreSQL driver pg;
- middleware авторизации, безопасности, ограничений доступа и трассировки;
- доменные сервисы в `services/`;
- API-маршруты в `routes/`.

Основные группы API:

- авторизация и пользователи;
- материалы, экземпляры материалов и свойства;
- рецепты;
- электролиты и сепараторы;
- ленты;
- партии электродов;
- аккумуляторы;
- проекты и доступ;
- отчёты;
- активность/трассировка;
- задел под обработку данных электрохимических испытаний и аналитику.

База данных:

- СУБД PostgreSQL;
- схема развивается только через forward-only SQL-миграции;
- в текущем комплекте ЕСПД v0.1.16 зафиксировано 49 SQL-файлов в `migrations/` и 49 SQL-файлов в `migrations_ASCII/`, включая миграции до `d040_add_coated_thickness_fields.sql`;
- текущее состояние применённых миграций контролируется таблицей `public.schema_migrations`; ожидаемый ledger после d040: `dima=21, dalia=28`;
- справочники и workflow-таблицы связаны внешними ключами и сервисной валидацией;
- часть защитных правил реализована на уровне БД, например проверка стека электродов аккумулятора.

Аналитика в текущей версии в основном расчётная и workflow-ориентированная:

- расчёт масс компонентов рецепта;
- расчёт параметров электродов по фактическим массам;
- расчётные ёмкости электродов и аккумуляторов;
- подбор/подсказки по стеку электродов;
- отдельный учёт технологических настроек и измерений, например зазоров нанесения и толщин после нанесения/сушки до каландрирования;
- печатные отчёты по объектам.

Расширенная обработка данных электрохимических испытаний, включая результаты циклирования аккумуляторов, и графическая аналитика относится к следующему этапу/V2, хотя базовые таблицы и задел уже присутствуют.

5. Безопасность, авторизация и контроль пользователей

В приложении реализована авторизация пользователей. Серверные маршруты проверяют токен пользователя и права доступа.

Основные принципы:

- обычные операции выполняются через авторизованный веб-интерфейс;
- `created_by` и `updated_by` являются серверными audit-полями, браузер не должен сам назначать автора записи;
- пароли хешируются;
- используется JWT-авторизация;
- предусмотрены роли пользователей и контроль доступа к пользовательским и административным операциям;
- для разработки существует `auth bypass`, но запуск в рабочей среде не должен разрешать `AUTH_BYPASS=true`; при промышленном размещении этот режим должен быть отключён конфигурационно и не использоваться в рабочей среде;
- критичные destructive-действия имеют проверки зависимостей и подтверждения;
- в базе сохраняются audit-поля и журналы для поддерживаемых сценариев активности/изменений; для промышленного контура требования к инфраструктурному журналированию нужно согласовать отдельно.

Для промышленного размещения потребуется дополнительно согласовать:

- способ выпуска и хранения секрета рабочей среды `JWT_SECRET`;
- HTTPS/TLS — шифрование веб-соединения между браузером и сервером через сертификат;
- правила резервного копирования и восстановления. В текущем Windows-пилоте предусмотрены PowerShell-скрипты для создания SQL-резервных копий, но для размещения на ресурсах Росатома нужен утверждённый регламент: расписание, место хранения, срок хранения, ответственное лицо и проверка восстановления;
- доступ администраторов к серверу и БД;
- сетевые ограничения/IP allowlist — доступ только из разрешённых сетей или с разрешённых адресов, если этого требуют политики площадки.

6. Аппаратные требования к инфраструктуре

Для текущего лабораторного пилота требования умеренные.

Минимально:

- ОС: Linux-сервер или Windows-сервер/рабочая станция, согласованные с ИТ;
- PostgreSQL 16 или совместимая версия;
- Node.js LTS;
- 2 CPU cores;
- 4 GB RAM;
- 20-50 GB дискового пространства на старте.

Рекомендуемо для устойчивой эксплуатации:

- 4 CPU cores;
- 8 GB RAM;
- отдельное дисковое пространство под каталог данных PostgreSQL, файловое хранилище приложения и резервные копии;
- регулярные резервные копии БД;
- мониторинг свободного места.

Рост хранилища зависит от объёма загружаемых файлов, отчётов и будущих данных электрохимических испытаний. Табличные лабораторные записи сами по себе занимают мало места; основной рост ожидается от вложений, исходных файлов испытаний и резервных копий.

7. Сетевая доступность и взаимодействие с базой

Для обычных пользователей достаточно доступа через браузер к веб-интерфейсу решения. Прямой доступ пользователей к PostgreSQL не требуется и нежелателен. Рабочий обмен файлами предполагается через веб-интерфейс: пользователь выбирает файл в браузере, прикрепляет его к записи, скачивает ранее загруженный файл или выгружает печатный отчёт.

Предпочтительный вариант:

- сервер с приложением размещён внутри инфраструктуры, одобренной ИТ/Гринатомом;
- пользователи подключаются к интерфейсу через браузер;
- если доступ из КСПД возможен через терминальный сервер/RDP, этого достаточно для пилотного использования;
- если политика площадки разрешает доступ к внутреннему HTTP/HTTPS-адресу решения с лабораторных устройств, это предпочтительно для работы в лаборатории: операторы смогут вводить данные с ноутбуков или планшетов непосредственно во время приготовления лент, нарезки электродов и сборки аккумуляторов;
- загрузка/скачивание файлов выполняется через веб-интерфейс приложения;
- прямой доступ к БД, серверным папкам и файловому хранилищу нужен только администраторам/сопровождению, а не обычным пользователям.

Доступ через терминальный сервер/RDP подходит для пилотного размещения, если пользователи могут загружать и скачивать файлы через веб-интерфейс. Этот способ менее удобен для работы с файлами, чем прямой доступ с лабораторного устройства, но он является рабочим вариантом для начального этапа.

Отдельная общая сетевая папка для обычных пользователей не требуется и менее предпочтительна, потому что в таком варианте файл легко теряет связь с конкретным материалом, лентой, партией электродов, аккумулятором или другим физическим объектом.

Прикрепляемые файлы могут включать исходные лабораторные файлы, вспомогательные Excel-таблицы, технические задания, протоколы и отчёты, если это разрешено политикой площадки. Для чувствительных документов необходимо дополнительно согласовать правила доступа, резервного копирования и сроков хранения. В любом случае резервное копирование решения должно охватывать не только PostgreSQL, но и файловое хранилище приложения.

На текущий момент прикрепление файлов реализовано не как общий файловый архив, а как привязка документа к конкретной сущности:

- **материалы / экземпляры материалов** — документы по источнику и партии поставки (например, паспорт, сертификат/паспорт качества, документы заказа и получения, фото упаковки), а также документы по свойствам материала (например, результаты аналитических измерений, подтверждающие удельную ёмкость, плотность и другие свойства);
- **электролиты** — файлы, относящиеся к конкретной записи электролита (паспорт, состав, протокол приготовления или аналитики, сопроводительные документы);
- **сепараторы** — файлы, относящиеся к конкретной записи сепаратора (паспорт/спецификация, партия, аналитические или входные документы);
- **аккумуляторы** — файлы и текстовые заметки по электрохимическим испытаниям конкретного аккумулятора; для модуля циклирования также предусмотрена загрузка исходных файлов измерений и хранение разобранных данных испытаний.

Ближайшее практическое расширение — добавить такие же привязанные файловые вложения для **электродных лент и партий электродов**, чтобы исходные Excel-файлы, протоколы, фотографии или другие сопровождающие документы можно было хранить не в отдельной общей папке, а рядом с соответствующей записью в системе.

Расширение системы до хранения других категорий документов, например проектной документации, официальных отчётов или технических заданий, возможно, но должно рассматриваться как отдельное проектное решение. Перед таким расширением нужно согласовать с руководством и ответственными службами: какие категории документов допустимо хранить, кто имеет доступ, как они резервируются, как долго хранятся и какие требования информационной безопасности применяются.

Прямой доступ к БД нужен только:

- администратору;
- разработчику/сопровождающему специалисту, в том числе для точечной диагностики и исправления данных через `psql` при согласованной административной процедуре;
- для миграций, резервного копирования, восстановления и диагностики.

8. Ответственность за разработку и администрирование

Предлагаемая схема ответственности:

- ответственный исполнитель, функциональный владелец и руководитель развития решения: Мараулайте Д.К. Эта роль сохраняется и для текущей vanilla v1, и для будущих версий: изменения workflow, правил данных и допуск версии к рабочему использованию проходят через функциональную проверку и приёмку Мараулайте Д.К.;
- соразработчик и технический исполнитель по отдельным модулям, Vue/V2 и инфраструктуре: Меняйлов Д.С. Его вклад в V2-фронтенд и расширенные модули может быть существенно больше, чем в vanilla v1, но V2 должен быть проверен на соответствие данным, workflow и требованиям текущей рабочей версии перед размещением на рабочем сервере;
- консультации по электрохимическим требованиям и будущей аналитике: Баталов Р.С. по согласованию и при наличии доступности;
- инфраструктурное сопровождение после размещения на ресурсах Росатома: ответственную сторону предлагается определить совместно с ИТ/Гринатомом.

Разработческое сопровождение включает:

- развитие функциональности;
- проверку корректности лабораторных workflow;
- выпуск миграций;
- контроль совместимости V2 с правилами vanilla v1 и проверку V2 перед допуском к рабочему использованию;
- сопровождение документации.

Инфраструктурное сопровождение должно включать:

- доступность сервера;
- резервное копирование;
- восстановление;
- обновления ОС/СУБД/Node.js по согласованному регламенту;
- контроль места на диске;
- управление секретами рабочей среды.

9. Предложения по организации инфраструктуры

Для начального пилота предлагается простой вариант:

1. Разместить решение как веб-приложение на внутреннем сервере/виртуальной машине, согласованной с ИТ/Гринатомом.
2. Установить PostgreSQL, Node.js LTS и приложение.
3. Ограничить обычных пользователей веб-доступом через браузер.
4. Не выдавать прямой доступ к PostgreSQL обычным пользователям.
5. Организовать регулярные резервные копии БД и прикрепляемых файлов.
6. Настроить HTTPS и секреты рабочей среды.

7. Запускать изменения схемы только через SQL-миграции.
8. До замены интерфейса на Vue/V2 сохранить vanilla v1 как рабочую пилотную версию.

Со стороны разработчиков для обсуждения с IT/Гринатомом важно зафиксировать только несколько практических моментов:

- где будет размещено приложение и PostgreSQL;
- каким будет пользовательский доступ: через терминальный сервер/RDP, внутренний HTTP/HTTPS-адрес или оба варианта;
- кто отвечает за серверное администрирование, резервное копирование, восстановление и секреты рабочей среды;
- какие правила площадки применяются к хранению прикрепляемых файлов и чувствительных документов.